

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA HỆ THỐNG THÔNG TIN**



UIT

BÁO CÁO TỔNG KẾT

ĐỒ ÁN MÔN HỌC

**Đề tài: Thiết kế, xây dựng hệ thống quản lý cơ sở dữ liệu
cho Khách sạn – Phòng – Khách – Đặt phòng.**

Sinh viên thực hiện:

Nguyễn Trọng Nhân - 24521236

Trần Quang Trường - 24521901

Lương Huỳnh Gia Bảo - 24520006

Giảng viên hướng dẫn

ThS. Mai Văn Cường

BÁO CÁO ĐỒ ÁN CUỐI KHÓA
HỌC PHẦN: NHẬP MÔN CƠ SỞ DỮ LIỆU

I. THÔNG TIN SINH VIÊN / NHÓM SINH VIÊN

Sinh viên 1 – Họ và tên: Nguyễn Trọng Nhân

Mã số sinh viên: 24521236

Email: nguyentrongnhan06cm@gmail.com

Sinh viên 2 – Họ và tên: Trần Quang Trường

Mã số sinh viên: 24521901

Email: tranquangtruong039@gmail.com

Sinh viên 3 – Họ và tên: Lương Huỳnh Gia Bảo

Mã số sinh viên: 24520006

Email: 24520006@gm.uit.edu.vn

Số điện thoại liên hệ chung: 0947523907- Nguyễn Trọng Nhân

Ngành học: ATTT + KHMT

Tên đề thi / đề bài được giao: Khách sạn – Phòng – Khách – Đặt phòng

Mục Lục

BÁO CÁO ĐỒ ÁN CUỐI KHÓA	1
HỌC PHẦN: NHẬP MÔN CƠ SỞ DỮ LIỆU.....	1
I. THÔNG TIN SINH VIÊN / NHÓM SINH VIÊN	1
II. BẢNG PHÂN CÔNG NHIỆM VỤ	3
A. Nhiệm vụ của các thành viên.....	3
B. Nhận xét.....	3
III. NỘI DUNG BÁO CÁO	4
1. Giới thiệu đề tài.....	4
2. Cấu trúc lược đồ quan hệ	4
3. Thiết kế và cài đặt CSDL (DDL)	10
4. Cài đặt dữ liệu mẫu (DML).....	13
5. Truy vấn Đại số quan hệ.....	19
6. Truy vấn SQL	21
7. View.....	26
8. Trigger và ràng buộc toàn vẹn.....	29
9. Phân quyền và bảo mật.....	33
10. Kết luận	35

II. BẢNG PHÂN CÔNG NHIỆM VỤ

A. Nhiệm vụ của các thành viên.

MSSV	Họ và Tên	Nhiệm vụ	Vai trò
24521236	Nguyễn Trọng Nhân	Phân công, điều phối các bạn, chia câu hỏi ra cho nhóm, tạo server (kết hợp ngrok với PostgreSQL), thực hiện câu 1,2,3,4,5,6 (các ý chính, scripts, hình ảnh, charts- Mermaid).	Trưởng nhóm.
24521901	Trần Quang Trường	Thực hiện các câu 7, hoàn chỉnh các câu 3,4,5,6,7,8, chỉnh sửa văn phong, chỉnh sửa scripts cho các câu, kiểm tra đánh giá và hoàn thiện đồ án.	Quản lý + Kiểm tra, đánh giá đồ án.
24520006	Lương Huỳnh Gia Bảo	Thực hiện các câu 8,9, 10, hỗ trợ nhóm lên ý tưởng, tổng hợp các kiến thức, đề xuất nội dung từng phần cho nhóm.	Thực hiện.

B. Nhận xét

- **Nguyễn Trọng Nhân:** Hoàn thành được các nhiệm vụ mà thầy giao phó, Cần quan tâm các bạn nhiều hơn, kiểm tra các nội dung trong đồ án một cách chặt chẽ hơn, rà soát lại thông tin cho toàn bộ nhóm, đưa ra thời gian làm việc hợp lý, học được cách kết nối mọi người, có được khả năng làm việc nhóm hiệu quả.
- **Trần Quang Trường:** Hoàn thành tốt được nhiệm vụ mà thầy, nhóm trưởng giao phó, năng nổ, cầu toàn, chín chu, phối hợp tốt với các bạn trong nhóm.
- **Lương Huỳnh Gia Bảo:** Hoàn thành tốt nhiệm vụ được giao phó, thông minh, sáng tạo trong các làm việc, vui vẻ, hoạt bát, năng nổ trong công việc.

III. NỘI DUNG BÁO CÁO

1. Giới thiệu đề tài

Trong bối cảnh sự phát triển của các ngành dịch vụ nói chung và các dịch vụ lưu trú nói riêng, việc lưu trữ, kiểm soát dữ liệu quản lý phòng, khách hàng và hoạt động đặt phòng là vô cùng cần thiết.

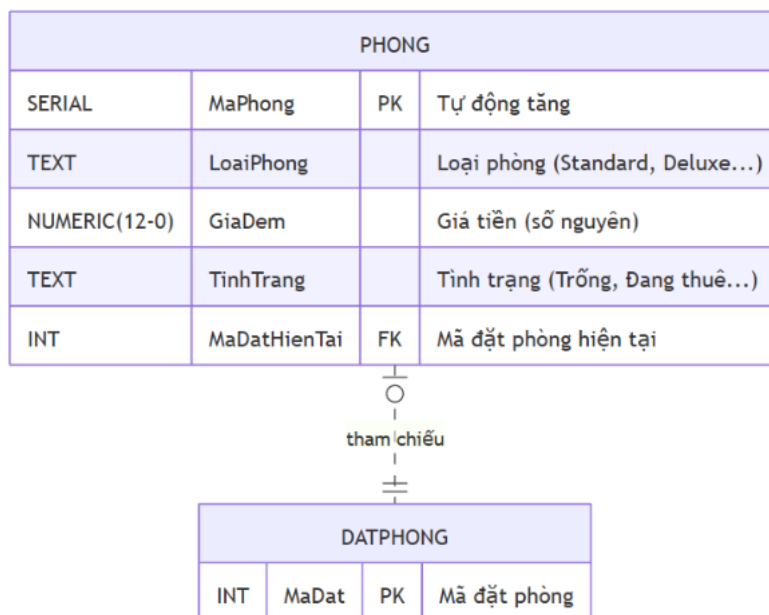
Chúng em cung cấp giải pháp - hệ thống quản lý khách sạn dựa trên mô hình cơ sở dữ liệu quan hệ. Hệ thống tập trung vào thực thể chính là phòng, khách và đặt phòng.

Mục tiêu chính của đề tài là:

- Thiết kế và triển khai một cơ sở dữ liệu quan hệ hoàn chỉnh trên hệ quản trị PostgreSQL.
- Xây dựng các bên liên quan:
- Lược đồ quan hệ.
- Ràng buộc khóa chính - khóa ngoại.
- Đại số quan hệ.
- Các thao tác dữ liệu (DDL,DML) .

2. Cấu trúc lược đồ quan hệ

2.1 Bảng PHONG lưu trữ thông tin về các phòng trong khách sạn.



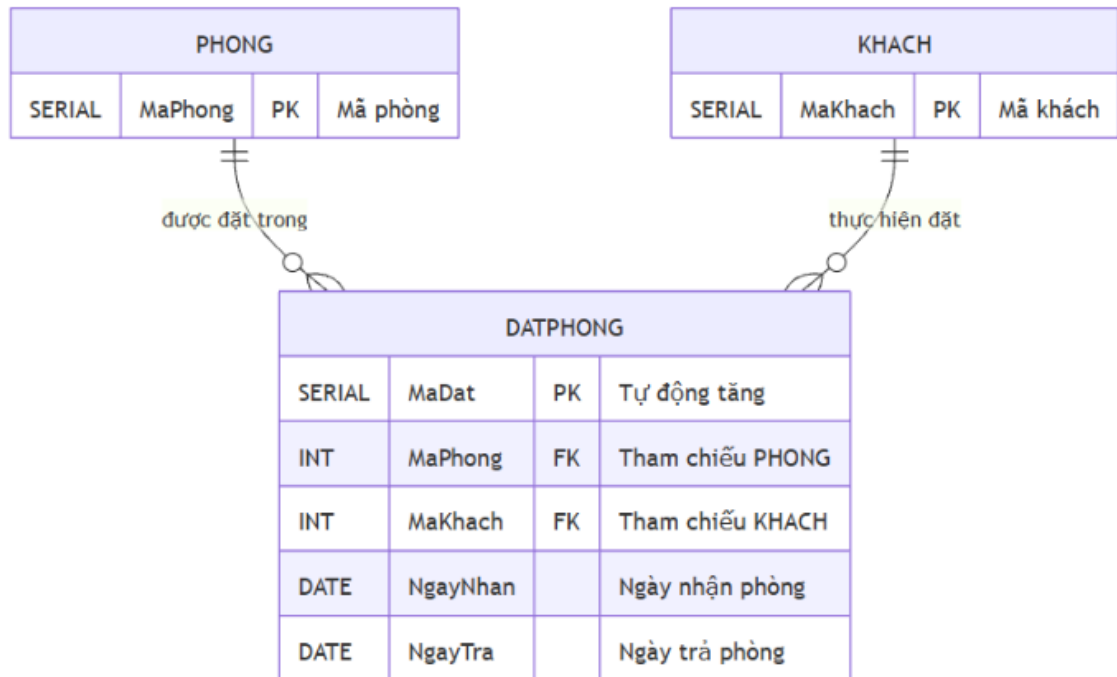
- **Khóa chính (Primary Key):** MaPhong
- **Khóa ngoại (Foreign Key):**
 - MaDatHienTai tham chiếu đến DATPHONG(MaDat)

2.2 Bảng KHACH lưu trữ thông tin cá nhân của khách hàng sử dụng dịch vụ khách sạn.

KHACH			
SERIAL	MaKhach	PK	Mã khách (Tự tăng)
TEXT	HoTen		Họ tên (NOT NULL)
VARCHAR(12)	CCCD		Căn cước (UNIQUE)
VARCHAR(15)	SDT		Số điện thoại

- **Khóa chính (Primary Key):** MaKhach
- Bảng KHACH không chứa khóa ngoại, nhưng được các bảng khác tham chiếu đến.

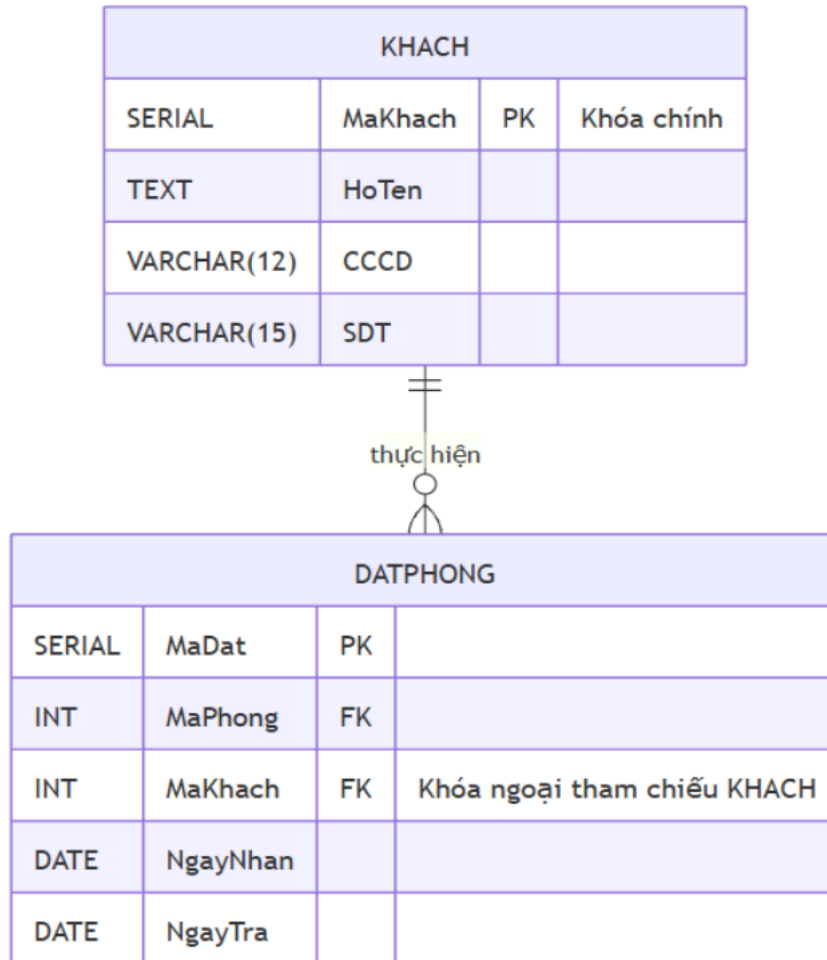
2.3 Bảng DATPHONG lưu trữ thông tin các lượt đặt phòng của khách hàng.



- **Khóa chính (Primary Key):** MaDat
- **Khóa ngoại (Foreign Key):**
 - MaPhong tham chiếu đến PHONG(MaPhong)
 - MaKhach tham chiếu đến KHACH(MaKhach)

2.4 Mối liên hệ giữa các bảng

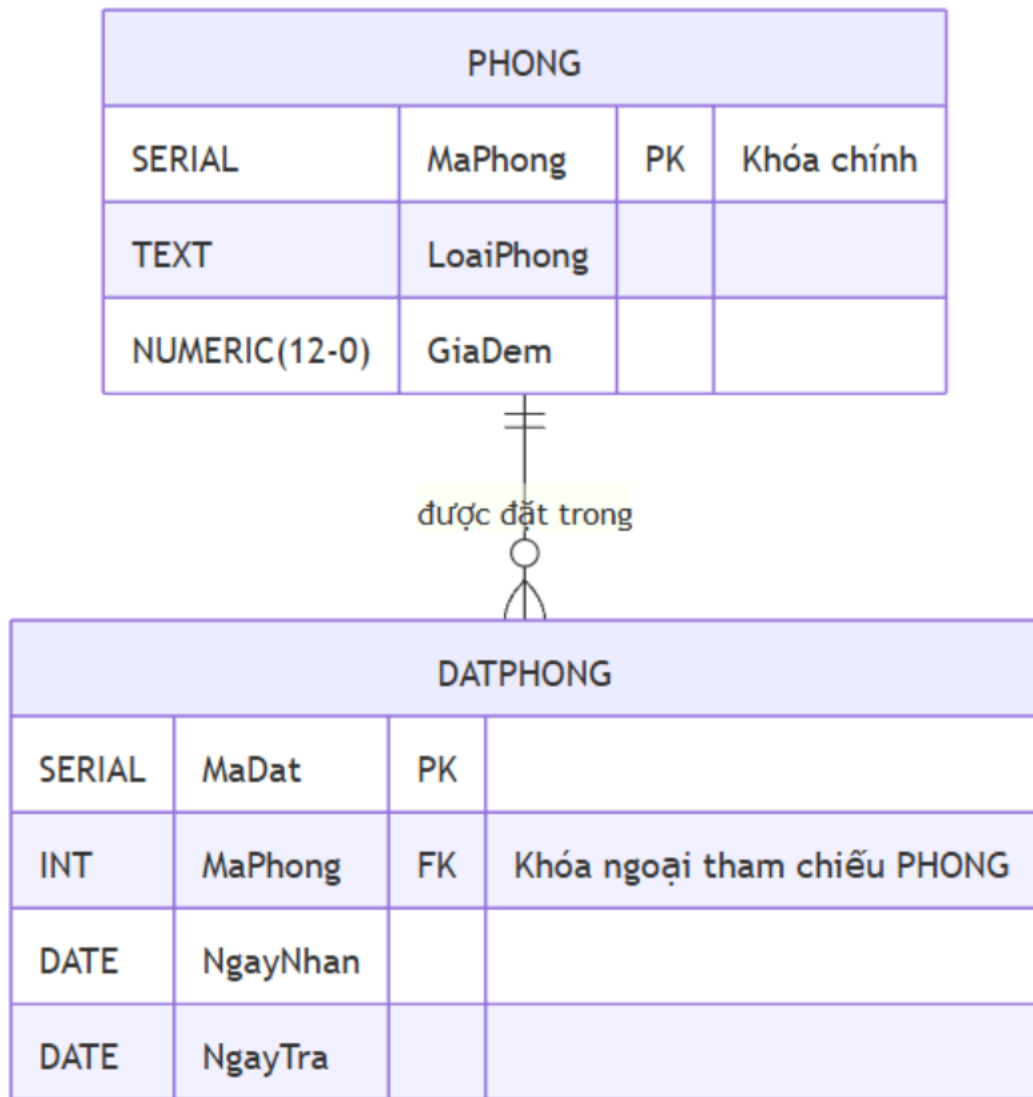
Mối liên hệ giữa KHACH – DATPHONG:



Quan hệ **một – nhiều (1–N)**.

Một khách hàng có thể thực hiện nhiều lượt đặt phòng, nhưng mỗi lượt đặt phòng chỉ thuộc về một khách hàng.

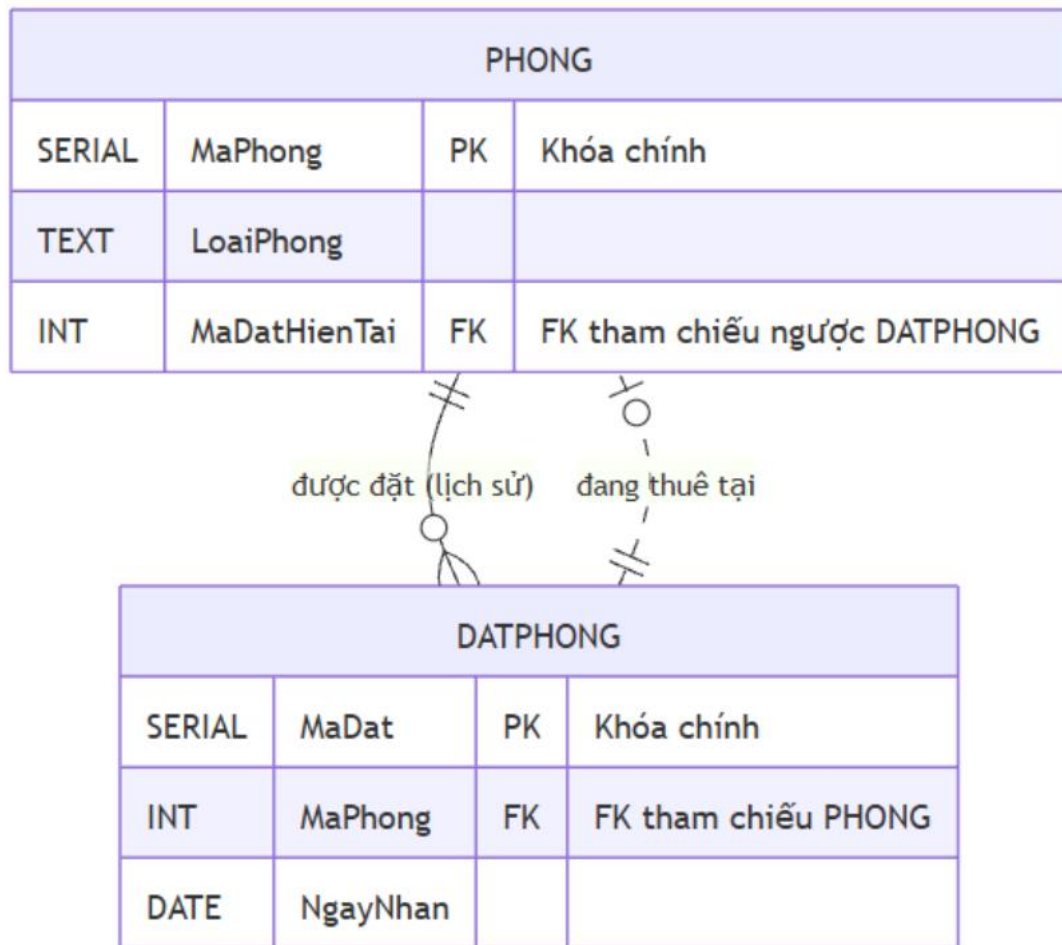
Mối liên hệ giữa PHONG – DATPHONG:



Quan hệ **một – nhiều (1–N)**.

Một phòng có thể được đặt nhiều lần tại các thời điểm khác nhau, nhưng mỗi lượt đặt phòng chỉ gắn với một phòng cụ thể.

Mối liên hệ giữa PHONG – DATPHONG (tham chiếu lẫn nhau):



Ngoài khóa ngoại DATPHONG.MaPhong → PHONG.MaPhong, bảng PHONG còn có thuộc tính MaDatHienTai tham chiếu ngược lại DATPHONG.MaDat.

Điều này tạo thành **cặp tham chiếu lẫn nhau (mutual foreign key)**, dùng để xác định mã đặt phòng hiện tại của mỗi phòng tại một thời điểm.

3. Thiết kế và cài đặt CSDL (DDL)

- Trình bày các câu lệnh tạo bảng.
- Mô tả việc vô hiệu hóa ràng buộc khai báo khi thay thế bằng trigger (nếu có).

3.1 Bảng **KHACH**

KHACH(MaKhach PK, HoTen, CCCD, SDT)

```
CREATE TABLE KHACH (  
    MaKhach SERIAL PRIMARY KEY,  
    HoTen TEXT NOT NULL,  
    CCCD VARCHAR(12) UNIQUE NOT NULL,  
    SDT VARCHAR(10) NOT NULL,  
    CONSTRAINT chk_cccd CHECK (CCCD ~ '^[0-9]{12}$'),  
    CONSTRAINT chk_sdt CHECK (SDT ~ '^[0-9]{10}$')  
);
```

Bảng **KHACH** lưu trữ thông tin khách hàng:

- MaKhach là khóa chính, được sinh tự động.
- Thuộc tính CCCD được khai báo là UNIQUE nhằm đảm bảo mỗi căn cước chỉ thuộc về một khách hàng.
- Ngoài ra, thuộc tính CCCD và SDT được thêm ràng buộc CHECK nhằm đảm bảo mỗi căn cước có chính xác 12 số và số điện thoại có chính xác 10 số.

Có phần hơi khác ở đây là ở cột **MaKhach – PK** thì chúng em có dùng **SERIAL** để triển khai tự động hóa sinh khóa chính cho mỗi bản ghi, giúp tránh trùng lặp khóa, giảm lỗi khi nhập dữ liệu và phù hợp với hệ thống có nhiều người dùng đồng thời, không cần chủ động tạo mã khách hàng mà SERIAL sẽ giúp ta làm việc này.

3.2 Bảng *PHONG*

PHONG(MaPhong PK, LoaiPhong, GiaDem, TinhTrang, MaDatHienTai FK
→ **DATPHONG**.MaDat)

```
CREATE TABLE PHONG (  
  MaPhong SERIAL PRIMARY KEY,  
  LoaiPhong TEXT NOT NULL DEFAULT 'Standard',  
  GiaDem NUMERIC(12,0) NOT NULL CHECK (GiaDem >= 0),  
  TinhTrang TEXT NOT NULL,  
  MaDatHienTai INT,  
  CONSTRAINT chk_loaiphong CHECK (LoaiPhong IN ('Suite','Deluxe','Standard')),  
  CONSTRAINT chk_tinhtrang CHECK (TinhTrang IN ('TRONG','DANG_THUE','BAO_TRI'))  
);
```

Bảng **PHONG** lưu trữ thông tin các phòng trong khách sạn.

- **MaPhong** là khóa chính.
- Thuộc tính **MaDatHienTai** dùng để lưu mã đặt phòng hiện tại của phòng và sẽ được khai báo khóa ngoại sau do tồn tại tham chiếu lẫn nhau với bảng **DATPHONG**.

3.3 Bảng **DATPHONG**

DATPHONG(MaDat PK, MaPhong FK → **PHONG**.MaPhong, MaKhach FK
→ **KHACH**.MaKhach, NgayNhan, NgayTra)

```
CREATE TABLE DATPHONG (
MaDat SERIAL PRIMARY KEY,
MaPhong INT,
MaKhach INT,
NgayNhan DATE NOT NULL,
NgayTra DATE NOT NULL,
CONSTRAINT chk_ngay_thue CHECK (NgayTra >= NgayNhan),
CONSTRAINT fk_ma_phong
FOREIGN KEY (MaPhong) REFERENCES PHONG(MaPhong),
CONSTRAINT fk_ma_khach
FOREIGN KEY (MaKhach) REFERENCES KHACH(MaKhach)
);
```

Bảng **DATPHONG** lưu trữ thông tin các lượt đặt phòng.

- MaDat là khóa chính.
- MaPhong là khóa ngoại tham chiếu đến bảng **PHONG**.
- MaKhach là khóa ngoại tham chiếu đến bảng **KHACH**.

3.4 Bổ sung ràng buộc khóa ngoại tham chiếu lẫn nhau

Do tồn tại mối quan hệ tham chiếu lẫn nhau giữa bảng **PHONG** và **DATPHONG**, khóa ngoại **MaDatHienTai** của bảng **PHONG** được bổ sung sau khi cả hai bảng đã được tạo:

```
ALTER TABLE PHONG
ADD CONSTRAINT fk_phong_dat
FOREIGN KEY (MaDatHienTai) REFERENCES DATPHONG(MaDat);
```

Cách cài đặt này giúp tránh lỗi phụ thuộc vòng khi tạo bảng trong PostgreSQL, đồng thời đảm bảo đúng mối liên hệ giữa các bảng theo lược đồ quan hệ đã thiết kế.

Chú thích:

- **ALTER TABLE** dùng để thay đổi cấu trúc của một bảng đã tồn tại, ta không tạo bảng mới, mà bổ sung ràng buộc cho bảng **PHONG** sau khi nó đã được tạo
- **ADD CONSTRAINT**: thêm một ràng buộc mới, ta đặt tên theo ý mình không thì PostgreSQL tự sinh ra rất dài, khó nhớ.

4. Cài đặt dữ liệu mẫu (DML)

- Trình bày cách thêm dữ liệu vào các bảng.
- Mô tả thứ tự thêm dữ liệu trong trường hợp có tham chiếu lẫn nhau.

Nhiệm vụ của chúng ta:

- Đảm bảo số lượng bản ghi đúng:
- Bảng KHACH: khoảng 50 dòng.
- Các bảng có tham chiếu lẫn nhau (**PHONG,DATPHONG**): tối thiểu 40 dòng mỗi bảng.
- Đảm bảo toàn vẹn khóa ngoại, đặc biệt trong trường hợp tồn tại **tham chiếu lẫn nhau (mutual foreign key)** giữa bảng **PHONG** và **DATPHONG**.
- Đảm bảo dữ liệu đủ đa dạng để tất cả các truy vấn đại số quan hệ và SQL phía sau đều có kết quả trả về ít nhất một dòng.

Do tồn tại các cặp tham chiếu lẫn nhau:

- DATPHONG.MaPhong → PHONG.MaPhong
- PHONG.MaDatHienTai → DATPHONG.MaDat

nên không thể chèn dữ liệu đầy đủ cho cả hai bảng ngay từ đầu mà không vi phạm ràng buộc khóa ngoại.

Nên ta sẽ thực hiện lần lượt các bước sau:

- Chèn dữ liệu vào bảng KHACH trước vì nó độc lập.
- Chèn dữ liệu vào bảng PHONG với cột MaDatHienTai = NULL.
- Chèn dữ liệu vào bảng DATPHONG: tham chiếu đến PHONG và KHACH.
- Cập nhật lại PHONG.MaDatHienTai sau khi bảng DATPHONG đã có dữ liệu tương ứng.

Trong PostgreSQL có hỗ trợ một hàm - generate_series:

```
it002=> SELECT * FROM generate_series(1, 5);
```

```
=====
```

```
generate_series
```

```
-----
```

```
1
```

```
2
```

```
3
```

```
4
```

```
5
```

```
(5 rows)
```

4.1 Thêm dữ liệu cho bảng KHACH

```

it002=> INSERT INTO KHACH(HoTen, CCCD, SDT)
SELECT
    ho || ' ' || dem || ' ' || ten AS HoTen,
    ('02'
        || lpad(gen_x::text, 4, '0')
        || lpad(gen_y::text, 4, '0')
        || lpad(gen_z::text, 2, '0')
    )::varchar(12) AS CCCD,
    ('09'
        || lpad(gen_x::text, 4, '0')
        || lpad(gen_y::text, 4, '0')
    )::varchar(10) AS SDT
FROM (
    SELECT
        i,
        (ARRAY['Nguyen','Tran','Le','Pham','Hoang','Vu','Do','Bui','Dang'])[(i % 9) + 1] AS ho,
        (ARRAY['Van','Thi','Minh','Quang','Thu','Thanh'])[(i % 6) + 1] AS dem,
        (ARRAY['Nam','Anh','Lan','Huy','Ha','Tuan','Kiet','Mai','Long'])[(i % 9) + 1] AS ten,
        ((1329 * i) % 10000) AS gen_x,
        ((9817 * i) % 10000) AS gen_y,
        (i % 100) AS gen_z
    FROM generate_series(1, 50) AS i
) t;
SELECT * FROM KHACH LIMIT 5;
=====
INSERT 0 50

```

makhach	hoten	cccd	sdt
1	Tran Thi Anh	021329981701	0913299817
2	Le Minh Lan	022658963402	0926589634
3	Pham Quang Huy	023987945103	0939879451

4	Hoang Thu Ha	025316926804	0953169268
5	Vu Thanh Tuan	026645908505	0966459085
(5 rows)			

Ta sẽ sinh ngẫu nhiên họ tên từ một tập Họ, Tên lót và Tên mà ta định nghĩa trước.

CCCD và SDT cũng được sinh ngẫu nhiên theo công thức:

- $CCCD = 02 + gen_x + gen_y$
- $SDT = gen_x + gen_y$

Với gen_x và gen_y là 2 bộ số ngẫu nhiên trong khoảng $[0, 9999]$ theo một công thức:

- $gen_x = (1329 * i) \% 10000$
- $gen_y = (9817 * i) \% 10000$

Các số 1329 và 9817 nhằm tạo sự ngẫu nhiên cho tập dữ liệu.

4.2 Thêm dữ liệu cho bảng **PHONG**

```

it002=> INSERT INTO PHONG(LoaiPhong, GiaDem, TinhTrang, MaDatHienTai)
SELECT
    CASE WHEN i%3=0 THEN 'Suite'
    WHEN i%3=1 THEN 'Deluxe'
    ELSE 'Standard' END,
    800000 + i*10000,
    CASE WHEN i%20=0 THEN 'BAO_TRI'
    ELSE 'TRONG' END,
    NULL
FROM generate_series(1,50) i;
SELECT * FROM PHONG LIMIT 5;
=====
INSERT 0 50
maphong | loaiphong | giadem | tinhtrang | madathientai
-----+-----+-----+-----+-----
1 | Deluxe   | 810000 | TRONG    |
2 | Standard | 820000 | TRONG    |

```

3	Suite	830000	TRONG	
4	Deluxe	840000	TRONG	
5	Standard	850000	TRONG	(5 rows)

Loại phòng thuộc 1 trong 3 loại Suite, Deluxe, và Standard.

Giá sẽ được sinh với công thức tính là $800000 + i * 10000$

Tình trạng phụ thuộc vào việc MaDatHienTai có giá trị hay không, nên ta mặc định gán là TRONG. Tuy nhiên để thêm chút thực tế, ta sẽ gán một số phòng là BAO_TRI. MaDatHienTai được gán NULL để tránh vi phạm khóa ngoại khi bảng DATPHONG chưa có dữ liệu.

4.3 Thêm dữ liệu cho bảng DATPHONG

```

it002=> INSERT INTO DATPHONG(MaPhong, MaKhach, NgayNhan, NgayTra)
SELECT
    (1 + floor(random()*50)::int),
    (1 + floor(random()*50)::int),
    (DATE '2026-01-01' + i * 6) AS NgayNhan,
    (DATE '2026-01-01' + i * 6) + (1 + floor(random()*5)::int) AS NgayTra
FROM generate_series(1, 100) AS i;
SELECT * FROM DATPHONG LIMIT 5;
INSERT 0 100
madat | maphong | makhach | ngaynhan | ngaytra
-----+-----+-----+-----+-----
1 | 16 | 35 | 2026-01-07 | 2026-01-08
2 | 43 | 27 | 2026-01-13 | 2026-01-14
3 | 13 | 49 | 2026-01-19 | 2026-01-22
4 | 32 | 31 | 2026-01-25 | 2026-01-28
5 | 21 | 50 | 2026-01-31 | 2026-02-04
(5 rows)

```

MaPhong và MaKhach được sinh ngẫu nhiên từ 1 tới 50 để tạo được những trường hợp một phòng được đặt bởi nhiều người vào thời điểm khác nhau và trường hợp một người đặt nhiều phòng khác nhau.

Vì MaPhong và MaKhach được sinh ngẫu nhiên, nên ta cũng cần đảm bảo việc một phòng được đặt bởi nhiều người khác nhau phải được đặt ở khoảng thời gian khác nhau. Để đảm bảo điều đó, ta sẽ sinh khoảng thời gian không giao nhau với công thức:

- $\text{NgayNhan} = ('2026-01-01' + i * 6)$
- $\text{NgayTra} = \text{NgayNhan} + \text{random}(1, 5).$

4.4 Cập nhật lại MaDatHienTai cho bảng PHONG.

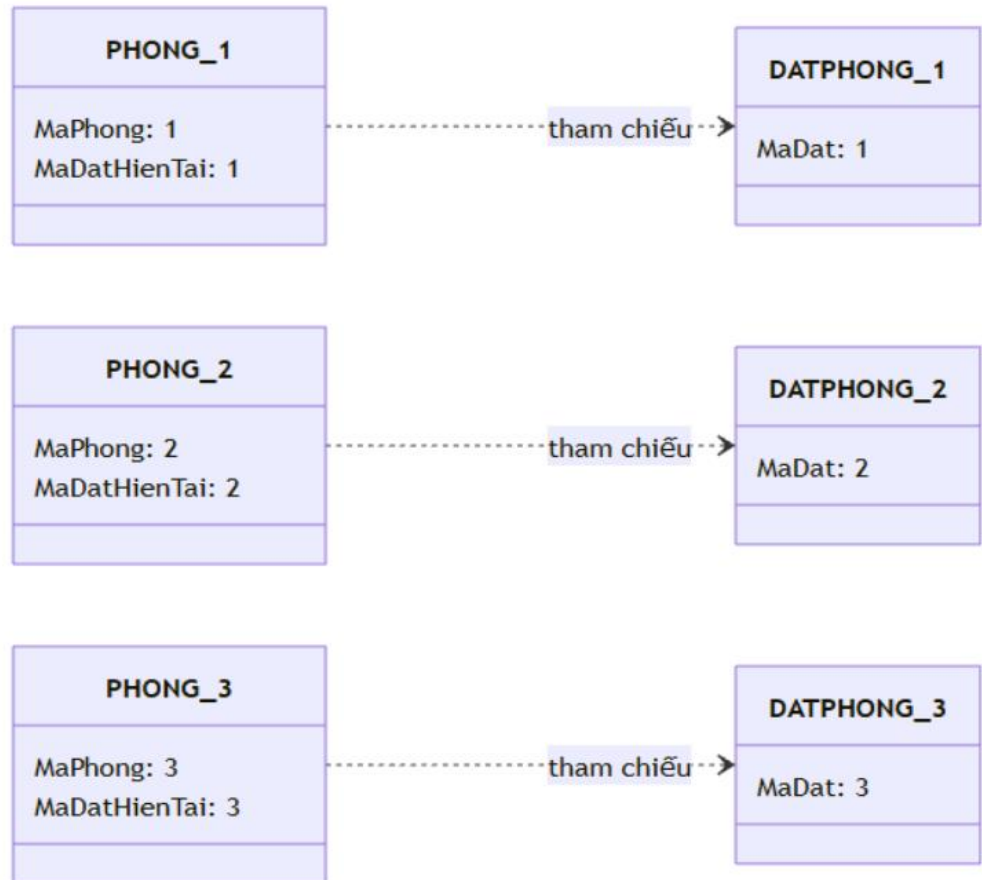
```
it002=> UPDATE PHONG p
SET
    MaDatHienTai = d.MaDat,
    TinhTrang = 'DANG_THUE'
FROM (
    SELECT DISTINCT ON (MaPhong)
        MaPhong, MaDat
    FROM DATPHONG
    ORDER BY MaPhong, NgayNhan ASC, MaDat ASC
) d
WHERE p.TinhTrang = 'TRONG' AND p.MaPhong = d.MaPhong;

===

UPDATE 38
```

Sau khi DATPHONG đã có dữ liệu, mỗi phòng ứng với mã đặt tương ứng đều có MaDatHienTai và TinhTrang là **'DANG_THUE'**.

/// ...



TRẠNG THÁI SAU ĐÓ: Đã thiết lập tham chiếu (FK)

5. Truy vấn Đại số quan hệ

5.1 Truy vấn 1: Liệt kê các phòng có giá/dêm $\geq 1.000.000$.

Điều kiện chọn σ : GiaDem ≥ 1000000

Phép chiếu π : Lấy ra các thuộc tính cần thiết (MaPhong, LoaiPhong, GiaDem, TinhTrang).

$$\pi_{MaPhong, LoaiPhong, GiaDem, TinhTrang}(\sigma_{GiaDem \geq 1000000}(PHONG))$$

5.2 Truy vấn 2: Tìm các khách đã đặt phòng ít nhất 3 lần.

- Cần kết nối bảng KHACH và DATPHONG để lấy thông tin khách và số lần đặt:

$$R_1 \leftarrow KHACH \bowtie DATPHONG$$

- Sử dụng phép gom nhóm theo MaKhach và HoTen, đếm số lượng MaDat.

$$R_2 \leftarrow \gamma_{MaKhach, HoTen} \gamma_{count(MaDat) \rightarrow SoLanDat}(R_1)$$

- Áp dụng điều kiện chọn trên kết quả đếm được :

$$KQ \leftarrow \pi_{MaKhach, HoTen, SoLanDat}(\sigma_{SoLanDat \geq 3}(R_2))$$

•

Tổng kết:

$$\pi_{MaKhach, HoTen, SoLanDat}(\sigma_{SoLanDat \geq 3}(\gamma_{MaKhach, HoTen} \gamma_{count(MaDat) \rightarrow SoLanDat}(KHACH \bowtie DATPHONG)))$$

5.3 Truy vấn 3: Với mỗi phòng, cho biết mã đặt hiện tại của phòng đó.

Dùng kí hiệu τ để biểu diễn sự sắp xếp:

$$\tau_{MaPhong}(\pi_{MaPhong, MaDatHienTai}(PHONG))$$

6. Truy vấn SQL

6.1 Thống kê số lượt đặt theo phòng (MaPhong, LoaiPhong, SoLuotDat).

```
it002=> SELECT p.MaPhong, p.LoaiPhong, COUNT(d.MaDat) AS SoLuotDat
FROM PHONG p
LEFT JOIN DATPHONG d ON d.MaPhong = p.MaPhong
GROUP BY p.MaPhong, p.LoaiPhong;
```

===

maphong | loaiphong | soluotdat

-----+-----+-----

42 | Suite | 2

29 | Standard | 2

4 | Deluxe | 0

34 | Deluxe | 0

...

11 | Standard | 1

44 | Standard | 2

(50 rows)

Bảng PHONG p là bảng chính còn DATPHONG là bảng phát sinh.

- Ta thực hiện việc chọn LEFT JOIN 2 bảng này để đảm bảo mọi phòng đều xuất hiện trong kết quả kể cả khi chưa từng có lượt đặt (khi đó các cột từ DATPHONG là NULL).
- Ta gom nhóm dữ liệu theo MaPhong và LoaiPhong. Lúc này với mỗi nhóm (MaPhong, LoaiPhong), hàm tổng hợp COUNT(d.MaDat) chỉ đếm các giá trị khác NULL nên với các phòng không có bản ghi đặt sẽ cho SoLuotDat = 0.

6.2 Liệt kê 10 khách đặt phòng nhiều nhất (MaKhach, HoTen, SoLan).

```
it002=> SELECT k.MaKhach, k.HoTen, COUNT(d.MaDat) AS SoLan
FROM KHACH k
JOIN DATPHONG d ON k.MaKhach = d.MaKhach
GROUP BY k.MaKhach, k.HoTen
ORDER BY SoLan DESC
LIMIT 10;
```

===

makhach	hoten	solan
11	Le Thanh Lan	5
16	Bui Thu Mai	4
25	Bui Thi Mai	4
5	Vu Thanh Tuan	4
49	Hoang Thi Ha	4
13	Hoang Thi Ha	3
2	Le Minh Lan	3
9	Nguyen Quang Nam	3
50	Vu Minh Tuan	3
35	Dang Thanh Long	3

(10 rows)

- Bảng KHACH k là bảng chính (chứa thông tin khách), còn DATPHONG d là bảng phát sinh (mỗi dòng là một lần đặt phòng).
- Ta dùng JOIN giữa bảng KHACH k và bảng DATPHONG d để chỉ lấy các khách đặt phòng ít nhất 1 lần (Vì bài toán đang cần xếp hạng top 10 theo số lần đặt nên những khách chưa từng đặt sẽ không xuất hiện trong kết quả).

- Ta gom nhóm theo k.MaKhach và k.HoTen, khi đó mỗi nhóm tương ứng 1 khách; hàm tổng hợp COUNT(d.MaDat) sẽ đếm số bản ghi đặt phòng của khách đó và trả về cột SoLan.
- Cuối cùng, ta sắp xếp giảm dần theo SoLan để đưa khách đặt nhiều nhất lên đầu, rồi cắt dữ liệu bằng LIMIT 10 để chỉ lấy 10 dòng đầu tiên.

6.3 Liệt kê phòng kèm thông tin đặt hiện tại (MaPhong, LoaiPhong, MaDat, NgayNhan, NgayTra).

```
it002=> SELECT p.MaPhong, p.LoaiPhong, d.MaDat, d.NgayNhan, d.NgayTra
FROM PHONG p
JOIN DATPHONG d ON p.MaDatHienTai = d.MaDat;

===
maphong | loaiphong | madat | ngaynhan | ngaytra
-----+-----+-----+-----+-----
    16 | Deluxe   |    1 | 2026-01-07 | 2026-01-08
    43 | Deluxe   |    2 | 2026-01-13 | 2026-01-14
    13 | Deluxe   |    3 | 2026-01-19 | 2026-01-22
...
     8 | Standard |   87 | 2027-06-07 | 2027-06-09
    49 | Deluxe   |   88 | 2027-06-13 | 2027-06-16
(38 rows)
```

- Bảng PHONG p là bảng chính, còn DATPHONG d là bảng phát sinh.
 - Truy vấn thực hiện JOIN theo khóa tham chiếu p.MaDatHienTai = d.MaDat để ánh xạ MaDatHienTai của mỗi phòng sang bản ghi đặt tương ứng nhằm lấy ra các thuộc tính thời gian NgayNhan và NgayTra, từ đó ta sẽ biết phòng nào đang gắn với đơn đặt nào với thời gian thuê cụ thể.

6.4 Câu SQL tương ứng với phép chia của Đại số quan hệ:

Liệt kê các khách đã đặt tất cả các loại phòng mà khách “Nam” đã từng đặt.

```
it002=> SELECT k.MaKhach, k.HoTen
FROM KHACH k
WHERE NOT EXISTS (
    SELECT DISTINCT p1.LoaiPhong
    FROM DATPHONG d1
    JOIN PHONG p1 ON d1.MaPhong = p1.MaPhong
    JOIN KHACH k1 ON d1.MaKhach = k1.MaKhach
    WHERE k1.HoTen ILIKE '% Nam'
    EXCEPT
    SELECT DISTINCT p2.LoaiPhong
    FROM DATPHONG d2
    JOIN PHONG p2 ON d2.MaPhong = p2.MaPhong
    WHERE d2.MaKhach = k.MaKhach
);
```

```
====
makhach | hoten
-----+-----
      2 | Le Minh Lan
      5 | Vu Thanh Tuan
     13 | Hoang Thi Ha
     22 | Hoang Thu Ha
     25 | Bui Thi Mai
(5 rows)
```

Ta công thức như sau:

- Gọi A = tập hợp các LoaiPhong mà Nam đã đặt.

- Gọi $B(k)$ = tập các LoaiPhong mà Khách K đã đặt.

-> Ta cần các khách K sao cho:

$$A \subseteq B(k)$$

6.4.1 Khối A

```
SELECT DISTINCT p1.LoaiPhong
FROM DATPHONG d1
JOIN PHONG p1 ON d1.MaPhong = p1.MaPhong
JOIN KHACH k1 ON d1.MaKhach = k1.MaKhach
WHERE k1.HoTen ILIKE '% Nam'
```

Tạo ra tập A = các loại phòng mà khách có tên kết thúc bằng “Nam” đã từng đặt.

6.4.2 Khối B(k)

```
SELECT DISTINCT p2.LoaiPhong
FROM DATPHONG d2
JOIN PHONG p2 ON d2.MaPhong = p2.MaPhong
WHERE d2.MaKhach = k.MaKhach
```

Tạo ra tập $B(k)$ = các loại phòng mà khách k đã từng đặt.

6.4.3 Ghép lại

```
SELECT k.MaKhach, k.HoTen
FROM KHACH k
WHERE NOT EXISTS ( A EXCEPT B(k) );
```

- Vòng ngoài duyệt từng khách k trong bảng KHACH
- Điều kiện WHERE NOT EXISTS (...) kiểm tra:
 - Không tồn tại loại phòng nào mà Nam có nhưng k không có

Nếu đúng -> chọn khách k.

7. View

- Trình bày các view đã tạo và mục đích sử dụng.

VIEW 1: vw_phong_gia (Cho phép xem mã phòng, loại phòng và giá/đêm.)

```
CREATE OR REPLACE VIEW vw_phong_gia AS
SELECT
    MaPhong,
    LoaiPhong,
    GiaDem
FROM PHONG;
```

Kết quả:

it002=> SELECT * FROM vw_phong_gia LIMIT 5;

maphong | loaiphong | giadem

-----+-----+-----

4 | Deluxe | 840000

14 | Standard | 940000

20 | Standard | 1000000

24 | Suite | 1040000

28 | Deluxe | 1080000

(5 rows)

VIEW 2: vw_phong_so_luot_dat (Cho phép xem mã phòng, loại phòng kèm theo số lượt đặt của mỗi phòng.)

```
CREATE OR REPLACE VIEW vw_phong_so_luot_dat AS
SELECT
    p.MaPhong,
    p.LoaiPhong,
    COUNT(d.MaDat) AS SoLuotDat
FROM PHONG p
LEFT JOIN DATPHONG d
ON p.MaPhong = d.MaPhong
GROUP BY p.MaPhong, p.LoaiPhong;
```

Kết quả:

it002=> SELECT * FROM vw_phong_so_luot_dat LIMIT 5;

maphong | loaiphong | soluotdat

-----+-----+-----

42 | Suite | 2

29 | Standard | 2

4 | Deluxe | 0

34 | Deluxe | 0

41 | Standard | 5

VIEW 3: vw_chi_tiet_dat_phong (Cho phép xem mã đặt, họ tên khách, mã phòng, loại phòng, ngày nhận và ngày trả.)

```
CREATE OR REPLACE VIEW vw_chi_tiet_dat_phong AS
```

```
SELECT
```

```
    d.MaDat,
```

```
    k.HoTen,
```

```
    p.MaPhong,
```

```
    p.LoaiPhong,
```

```
    d.NgayNhan,
```

```
    d.NgayTra
```

```
FROM DATPHONG d
```

```
JOIN KHACH k
```

```
ON d.MaKhach = k.MaKhach
```

```
JOIN PHONG p
```

```
ON d.MaPhong = p.MaPhong;
```

Kết quả:

```
it002=> SELECT * FROM vw_chi_tiet_dat_phong LIMIT 5;
```

```
madat | hoten | maphong | loaiphong | ngaynhan | ngaytra
```

```
-----+-----+-----+-----+-----+-----
```

```
1 | Dang Thanh Long | 16 | Deluxe | 2026-01-07 | 2026-01-08
```

```
2 | Nguyen Quang Nam | 43 | Deluxe | 2026-01-13 | 2026-01-14
```

```
3 | Hoang Thi Ha | 13 | Deluxe | 2026-01-19 | 2026-01-22
```

```
4 | Hoang Thi Ha | 32 | Standard | 2026-01-25 | 2026-01-28
```

```
5 | Vu Minh Tuan | 21 | Suite | 2026-01-31 | 2026-02-04
```

(5 rows)

8. Trigger và ràng buộc toàn vẹn

TRIGGER 1 (trên DATPHONG): *Khi INSERT/UPDATE, NgayTra (nếu có) không được nhỏ hơn NgayNhan.*

Vì trigger này trùng với ràng buộc khai báo chk_ngay_thue ta đã tạo trước đó, nên trước khi tạo trigger này ta sẽ bỏ **CONSTRAINT chk_ngay_thue** của bảng **DATPHONG**:

```
ALTER TABLE DATPHONG DROP CONSTRAINT chk_ngay_thue;
```

Thay bằng TRIGGER để kiểm tra:

```
CREATE OR REPLACE FUNCTION check_ngay_dat_phong()
RETURNS TRIGGER AS $$
BEGIN
IF NEW.NgayTra < NEW.NgayNhan THEN
    RAISE EXCEPTION 'Lỗi: Ngày trả (%) không được nhỏ hơn ngày nhận (%)',
NEW.NgayTra, NEW.NgayNhan;
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_check_ngay
BEFORE INSERT OR UPDATE ON DATPHONG
FOR EACH ROW
EXECUTE FUNCTION check_ngay_dat_phong();
```

TRIGGER 2 (trên PHONG): Khi UPDATE MaDatHienTai, mã đặt được gán phải thuộc đúng phòng đó (DATPHONG.MaPhong = PHONG.MaPhong). Nếu không, từ chối thao tác

Ta sử dụng **TRIGGER** để kiểm tra

```
CREATE OR REPLACE FUNCTION check_madathientai_valid()
RETURNS TRIGGER AS $$
DECLARE
    v_MaPhong_TrongDatPhong INT;
BEGIN
    IF NEW.MaDatHienTai IS NOT NULL THEN
        SELECT MaPhong INTO v_MaPhong_TrongDatPhong
        FROM DATPHONG
        WHERE MaDat = NEW.MaDatHienTai;
        IF v_MaPhong_TrongDatPhong IS NULL OR
v_MaPhong_TrongDatPhong <> NEW.MaPhong THEN
            RAISE EXCEPTION 'Lỗi: Mã đặt % không thuộc về phòng %',
NEW.MaDatHienTai, NEW.MaPhong;
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_check_madathientai
BEFORE UPDATE OF MaDatHienTai ON PHONG
FOR EACH ROW
EXECUTE FUNCTION check_madathientai_valid();
```

TRIGGER 3 (trên KHACH): Không cho phép DELETE khách nếu khách đó đang được tham chiếu bởi DATPHONG.MaKhach.

Tương tự như trigger 1, đầu tiên ta bỏ ràng buộc về khóa ngoại **fk_ma_khach** của bảng **DATPHONG**:

```
ALTER TABLE DATPHONG DROP CONSTRAINT fk_ma_khach;
```

Ta sử dụng TRIGGER để kiểm tra

```
CREATE OR REPLACE FUNCTION check_delete_khach()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (SELECT 1 FROM DATPHONG WHERE MaKhach =
OLD.MaKhach) THEN
        RAISE EXCEPTION 'Lỗi: Không thể xóa khách hàng % vì đã có dữ liệu
đặt phòng.', OLD.MaKhach;
    END IF;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_check_delete_khach
BEFORE DELETE ON KHACH
FOR EACH ROW
EXECUTE FUNCTION check_delete_khach();
```

Nhận xét:

- Về hiệu quả: Với các quy tắc đơn giản để giữ cho khung dữ liệu chuẩn xác, dùng Constraint là tốt nhất vì nó giúp hệ thống chạy rất nhanh và ổn định.
- Về sự linh hoạt: Trigger lại rất mạnh khi gặp những yêu cầu khó, cần kiểm tra dữ liệu giữa nhiều bảng với nhau hoặc khi muốn hiện thông báo lỗi rõ ràng cho người dùng.

- Về việc sử dụng: Chúng ta không nên lạm dụng Trigger. Chỉ khi nào những ràng buộc có sẵn (Constraint) không giải quyết được vấn đề thì mới nên chuyển sang dùng Trigger để tránh làm máy chủ phải xử lý nặng nề không cần thiết.

Để kiểm tra các trigger đã cài đặt. Ta sử dụng DO ... EXCEPTION.

Lấy ví dụ kiểm tra thử việc INSERT vào DATPHONG một bản ghi có NgayTra < NgayNhan. Khi thực thi câu lệnh INSERT, trigger sẽ phát hiện dữ liệu không hợp lệ và ném ra lỗi (RAISE EXCEPTION). Khối lệnh kiểm thử sử dụng DO ... BEGIN ... EXCEPTION sẽ bắt lỗi này; nếu bắt được lỗi đúng như mong đợi thì in ra thông báo [PASS] (ngược lại, nếu INSERT vẫn chạy thành công thì in [FAIL] vì trigger không chặn được dữ liệu sai).

```
DO $$
BEGIN
  BEGIN
    INSERT INTO DATPHONG(MaPhong, MaKhach, NgayNhan, NgayTra)
    VALUES (1, 1, DATE '2026-01-10', DATE '2026-01-01');
    RAISE NOTICE '[FAIL][TRG1-INS-INVALID] Trigger không chặn dữ liệu sai';
  EXCEPTION WHEN OTHERS THEN
    RAISE NOTICE '[PASS][TRG1-INS-INVALID] %', SQLERRM;
  END;
END $$;
NOTICE: [PASS][TRG1-INS-INVALID] new row for relation "datphong" violates
check constraint "chk_ngay_thue"
DO
```

(Những trường hợp khác đã được kiểm tra chi tiết trong code.)

9. Phân quyền và bảo mật

- Trình bày các lệnh cấp và thu hồi quyền.
- User A: người tạo CSDL
- User B, C, D, E: nhận quyền từ A hoặc từ lẫn nhau

- CREATE ROLE A WITH LOGIN PASSWORD 'password_a';
- CREATE ROLE B WITH LOGIN PASSWORD 'password_b';
- CREATE ROLE C WITH LOGIN PASSWORD 'password_c';
- CREATE ROLE D WITH LOGIN PASSWORD 'password_d';
- CREATE ROLE E WITH LOGIN PASSWORD 'password_e';
- ALTER TABLE KHACH OWNER TO A;
- ALTER TABLE PHONG OWNER TO A;
- ALTER TABLE DATPHONG OWNER TO A;

Quyền cần thiết để thực hiện được tất cả 4 lệnh SQL và 3 trigger đã nêu trên:

- SELECT: đọc dữ liệu
- INSERT: thêm dữ liệu
- UPDATE: cập nhật dữ liệu
- DELETE: xóa dữ liệu

1) A cấp cho B (có *GRANT OPTION* - B có thể cấp quyền của bản thân cho người khác)

```
GRANT SELECT, INSERT, UPDATE, DELETE ON KHACH, PHONG,
DATPHONG to B WITH GRANT OPTION;
```

2) A cấp cho C (có *GRANT OPTION*)

```
GRANT SELECT, INSERT, UPDATE, DELETE ON KHACH, PHONG,
DATPHONG to C WITH GRANT OPTION;
```

3) C cấp cho D (có GRANT OPTION)

User C thực hiện:

```
GRANT SELECT, INSERT, UPDATE, DELETE ON KHACH, PHONG,  
DATPHONG to D WITH GRANT OPTION;
```

4) D cấp cho E

User D thực hiện:

```
GRANT SELECT, INSERT, UPDATE, DELETE ON KHACH, PHONG,  
DATPHONG to E;
```

5) B cấp cho E

User B thực hiện:

```
GRANT SELECT, INSERT, UPDATE, DELETE ON KHACH, PHONG,  
DATPHONG to E;
```

- User A thu hồi quyền đã cấp cho C với option CASCADE

```
REVOKE SELECT, INSERT, UPDATE, DELETE ON KHACH, PHONG,  
DATPHONG FROM C CASCADE;
```

Phân tích quyền của các user sau khi thực hiện REVOKE CASCADE.

- UserA: còn đầy đủ các quyền vì là Owner
- UserB: vì User A không thực hiện lệnh thu hồi đối với User B, nên B vẫn giữ nguyên các quyền SELECT, INSERT, UPDATE, DELETE và GRANT OPTION
- UserC: không còn quyền vì bị User A thu hồi quyền thông qua lệnh REVOKE
- UserD: không còn quyền vì D nhận quyền từ C. Khi A REVOKE ... FROM C CASCADE, tất cả quyền mà C cấp cho người khác cũng bị thu hồi
- UserE: còn đầy đủ quyền SELECT, INSERT, UPDATE, DELETE vì được cấp quyền từ B và D, mà B vẫn còn đầy đủ 2 các quyền

10. Kết luận

- Tổng kết kết quả thực hiện đồ án, nhóm đã thực hiện được:
- Thiết kế lược đồ quan hệ đầy đủ với 3 bảng chính: PHONG, KHACH, DATPHONG, cùng các mối quan hệ 1–n và khóa ngoại tham chiếu lẫn nhau.
- Triển khai câu lệnh DDL tạo bảng với đầy đủ ràng buộc PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE, và sử dụng ALTER TABLE để xử lý tham chiếu vòng.
- Cài đặt dữ liệu mẫu ngẫu nhiên với đủ đa dạng và số lượng để phục vụ truy vấn và kiểm thử tính đúng đắn.
- Xây dựng các truy vấn đại số quan hệ và SQL để thống kê, lọc, và phân tích dữ liệu thực tế.
- Tạo VIEW nhằm đơn giản hóa việc truy xuất thông tin quan trọng cho người dùng cuối.
- Áp dụng TRIGGER thay thế ràng buộc logic phức tạp, đảm bảo dữ liệu luôn nhất quán và chính xác khi phát sinh thay đổi.
- Thực hiện phân quyền và thu hồi quyền truy cập dữ liệu giữa các người dùng, tuân thủ nguyên tắc an toàn và bảo mật trong quản trị hệ thống.

III. CAM KẾT HỌC THUẬT

Chúng tôi cam kết rằng toàn bộ nội dung trong báo cáo này là kết quả của quá trình học tập và thực hiện đồ án của nhóm. Chúng tôi không sao chép, đạo văn hoặc sử dụng trái phép nội dung từ bất kỳ nguồn nào mà không ghi rõ nguồn tham khảo. Nếu vi phạm các quy định về học thuật, chúng tôi xin hoàn toàn chịu trách nhiệm theo quy định của Nhà trường.

Ngày 28 tháng 01 năm 2026

Đại diện nhóm sinh viên

Nhan

Nguyễn Trọng Nhân

